

# Software Integration & Maintenance

Fahim Foysal Abak

Lecture, Dept. of CSE

Dhaka International University

# Software Testing Strategies

**A testing strategy** is a well-planned approach that integrates test case design into structured steps for successful software development

**Provides a roadmap:**

- What to test and when to test
- Required time, effort, and resources

**Includes key activities:**

- Test planning
- Test case design
- Test execution
- Result collection and evaluation

# A Strategic Approach

- **Testing is planned and systematic**
  - Activities are defined in advance and executed step by step
- **Conduct formal technical reviews**
  - Identify errors early before actual testing begins
- **Start small, then expand**
  - Begin at the **component level**
  - Gradually move toward **full system integration**
- **Use different testing techniques**
  - Apply appropriate methods at different stages of development

# A Strategic Approach

- **Defined roles in testing**
  - Usually performed by **developers**
  - In large projects, handled by an **independent testing team**
- **Testing ≠ Debugging**
  - **Testing** → finds errors
  - **Debugging** → fixes errors
  - Debugging must be included in the overall strategy

# Software Testing: Verification & Validation (V&V)

- **Software testing is often referred to as V&V**
- **Verification (Building the product right)**
  - Ensures the software correctly implements required functions
  - Focus on **design and specifications**
  - Key question: **“Are we building the product right?”**
- **Validation (Building the right product)**
  - Ensures the software meets **customer requirements**
  - Focus on **user needs and expectations**
  - Key question: **“Are we building the right product?”**

# Software Testing: Verification & Validation (V&V)

**V&V include a wide range of Software Quality Assurance (SQA) activities**

- **Key Activities:**

- **Formal technical reviews** (early error detection)
- **Quality & configuration audits** (standards and version control)
- **Performance monitoring** (speed and efficiency)
- **Simulation** (testing in virtual environments)
- **Feasibility study** (practicality of the system)
- **Documentation review** (checking requirements and design docs)
- **Database review** (data structure and integrity)
- **Algorithm analysis** (correctness and efficiency of logic)
- **Testing types:**
  - **Development testing**
  - **Usability testing**
  - **Installation testing**

# Software Testing: Organizing for Testing

- **Psychological Perspective**
  - Software engineers (SEs) **analyze, model, and build programs** with documentation
  - From the SE's viewpoint, **testing feels destructive** because it tries to find flaws in their work
- **Common Misconceptions:**
  - **“Developers should not test”** → Incorrect
  - **“Software should be thrown over the wall to outsiders”** → Incorrect
  - **“Testers join only at the end”** → Incorrect
- **Reality:**
  - Testing should be **collaborative**
  - **Developers and testers work together**
  - Testing should **start early and continue throughout development**

# Strategy for Conventional Software Architecture

- **Unit Testing**
  - Focuses on **individual units/components** of the software as implemented in the source code
- **Integration Testing**
  - Focuses on **design and construction of the software architecture**
  - Ensures that units work together correctly
- **Validation Testing**
  - Ensures that **requirements and analysis** are correctly implemented in the software
  - Checks if the system meets **user expectations**
- **System Testing**
  - Tests the **software as a whole**, including all components and interfaces

# When is Testing Complete?

- **No definitive answer** – testing never truly ends
- **Every execution counts** – each time a user runs the software, it is being tested
- **Practical constraints** – testing often stops due to **time, budget, or resource limits**
- **Severity-based approach:**
  - Classify test results by **error severity levels**
  - Consider testing **complete** when:
  - Critical errors no longer occur
  - Major errors have been **repaired or eliminated**

# Software Testing: Strategic Issues

- **Specify product requirements early**
  - Requirements should be **quantifiable** before testing begins
- **Assess more than errors**
  - Include quality characteristics:
    - **Portability**
    - **Maintainability**
    - **Usability**
- **State testing objectives explicitly**
  - Objectives should be **measurable**, e.g.,
    - **Test effectiveness**
    - **Cost to find/fix defects**
    - **Test coverage**

# Software Testing: Strategic Issues

- **Understand the users**
  - Develop **user profiles** and **use-cases**
  - Focus testing on **actual usage scenarios**
- **Build self-testing software**
  - Use **anti-bugging techniques**
  - Support **automated testing** and **regression testing**

# Software Testing: Strategic Issues

- **Use formal technical reviews**
  - Acts as a **filter prior to testing**
  - Reduces overall **testing effort**
- **Continuous improvement**
  - Measure testing strategy performance
  - Use **metrics and statistical process control** for better quality

# Software Testing: Strategic Issues

- **Use formal technical reviews**
  - Acts as a **filter prior to testing**
  - Reduces overall **testing effort**
- **Continuous improvement**
  - Measure testing strategy performance
  - Use **metrics and statistical process control** for better quality

# What is Debugging?

## **Definition:**

Debugging is the process of identifying, analyzing, and removing errors (bugs) from a software system after testing.

## **Explanation:**

Debugging begins when a program produces incorrect results during testing. The developer must investigate why the error occurred and fix the underlying issue. It is often considered an *art* because it requires logical thinking, experience, and intuition rather than a fixed method.

# Debugging Process

The debugging process is a systematic approach to locate and fix the cause of a detected error.

- It starts when a test case is executed.
- The output is compared with the expected result.
- Any mismatch is called a **symptom** of an error.
- The actual cause of the problem is hidden.
- The goal is to connect the symptom with its root cause and fix it.

# Key Characteristics of Debugging

- ❑ Debugging happens after successful testing detects errors.
- ❑ It varies from person to person—some developers are naturally better at it.
- ❑ It involves logical reasoning and careful analysis.
- ❑ It requires understanding both the code and system design.

# Why Debugging is Difficult

Debugging is challenging due to several reasons:

- The **symptom and cause may be far apart** in the code.
- Fixing one bug may temporarily hide another bug.
- Some issues may not be real errors (e.g., rounding errors).
- Human mistakes can be difficult to trace.

# Why Debugging is Difficult

Additional challenges include:

- Errors caused by **timing issues** instead of logic errors.
- Difficulty in reproducing exact error conditions.
- Bugs that occur only occasionally (intermittent errors).
- Errors spread across multiple modules or processes.

# Debugging Strategies

## **Definition:**

Debugging strategies are structured methods used to identify and fix errors efficiently.

Debugging is not random—it uses a mix of logic, experience, and tools. There are three main strategies:

- **Brute Force**
- **Backtracking**
- **Cause Elimination**

# Brute Force

## **Definition:**

A simple debugging method that uses direct observation techniques to find errors.

- Uses print statements, memory dumps, or logs.
- Commonly used when no other method works.
- Easy to apply but often inefficient and time-consuming.

# Backtracking

## **Definition:**

A method that traces the program execution backward from the point where the error appears.

- Starts from the error location.
- Moves step-by-step backward through the code.
- Effective for small programs.
- Difficult for large systems due to many possible paths.

# Cause Elimination

## **Definition:**

A systematic method that eliminates possible causes until the real cause is found.

- Uses logical reasoning (induction and deduction).
- Collects data related to the error.
- Forms hypotheses and tests them.
- Eliminates incorrect causes step by step.
- Most efficient and structured approach.

**Thank You.**